Parshvanath Charitable Trust's
A. P. SHAH INSTITUTE OF TECHNOLOGY
(Approved by AICTE New Delhi & Govt. of Maharashtra, Affiliated to University of Mumbai)
(Religious Jain Minority)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(Artificial Intelligence & Machine Learning)

| | | | |
|---|---|---|---|
| **Academic Year:** | 2026-27 | **Name of Student:** | |
| **Semester:** | V | **Student ID:** | |
| **Class / Branch:** | TE – CSE (AI&ML) | **Roll No:** | |
| **Subject:** | WMD Lab | **Date of Submission:** | |

## Experiment No. 04

**Lab Outcome:** Develop responsive front-end applications using React components and state management.

**Aim**: Implement state management using Hooks and Context API; add form handling and validation.

**Software used**: JavaScript, NodeJS, React.

**Theory:** -

### 1. What is "State"?

Imagine a counter app. Initially, Count = 0. Click a button: Count = 1. Click again: Count = 2. The value 0 → 1 → 2 is changing information — in React, this changing information is called State.

> *Simple definition — State is data that can change while the application is running. Examples: a counter value, a student's name, login status, dark/light mode, shopping cart items, whether a menu is open, or form input values.*

### 2. What is State Management?

Suppose a school application has many components:

```
App
 │
 ├── Header
 │     └── Login Status
 │
 ├── StudentProfile
 │     └── Student Name
 │
 ├── Courses
```

```
│    └── Course List
│
└── Footer
```

If the student's name needs to be used in several components, we need a way to store the data, change the data, share the data, and keep all components updated. This process is called State Management.

> *State Management = Storing, changing, and sharing changing data in a React application. React provides Hooks for managing state, and the Context API for sharing state between components.*

## 3. Hooks and useState()

Hooks are special functions provided by React that allow functional components to use React features such as state, side effects, context, and references.

| Hook | Purpose |
|---|---|
| useState() | Store and change state |
| useEffect() | Perform side effects |
| useContext() | Read data from Context |
| useRef() | Store a reference to an element |
| useReducer() | Manage complex state |

This experiment focuses mainly on useState() and useContext(). A minimal counter example:

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </div>
  );
}

export default Counter;
```

const [count, setCount] = useState(0); can be read as: count is the current value, setCount is the function used to change it, and 0 is the initial value:

```
Initial State (count = 0)
        │
        ▼
```

```
  User clicks button
        |
        ▼
  setCount(count + 1)
        |
        ▼
     count = 1
        |
        ▼
React re-renders component
        |
        ▼
  Screen displays 1
```

A beginner may ask: why not simply write count = count + 1;? Because React needs to know that the state has changed. setCount(count + 1); tells React: "The state has changed. Please update the screen."

## 4. Props, Data Flow, and Prop Drilling

State can be shared between a parent and a child component through props, for example:

```
import { useState } from "react";

function Child({ name }) {
  return <h2>Hello {name}</h2>;
}

function App() {
  const [studentName, setStudentName] = useState("Vijesh");

  return (
    <div>
      <Child name={studentName} />
    </div>
  );
}

export default App;
```

Data flows from App → Child. This is called one-way data flow. The difficulty arises in larger applications, where a deeply nested component may need data from a distant ancestor:

```
App
  | userName
  ▼
Header
  | userName
  ▼
Navigation
  | userName
  ▼
UserProfile
```

```
  | userName
  ▼
UserName
```

This is called Prop Drilling — data is passed through components that don't actually need it, purely so that it can reach a component further down the tree. This is where the Context API becomes useful.

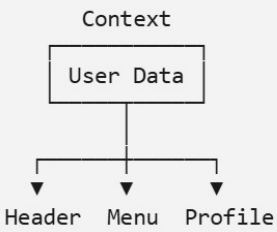
## 5. The Context API

Think of a school notice board: the principal posts a notice once, and students from any class can read it directly, without the principal telling each student individually. The Context API works the same way — it allows data to be shared with many components without passing props manually through every level.

Without Context (Prop Drilling):

```
App
  | props
  ▼
Header
  | props
  ▼
Menu
  | props
  ▼
Profile
```

With Context:




Context API has three main parts:

```
1. createContext()
        |
        ▼
2. Provider
        |
        ▼
3. useContext()
```

**Step 1 — createContext()**

```
import { createContext } from "react";
```

```
const UserContext = createContext();

export default UserContext;
```

This creates a shared space — UserContext — similar to a common notice board.

**Step 2 — Provider**

```
<UserContext.Provider value="Vijesh">
  <Profile />
</UserContext.Provider>
```

value="Vijesh" is the data being shared. Because Profile is nested inside the Provider, it can access that data.

**Step 3 — useContext()**

```
import { useContext } from "react";
import UserContext from "./UserContext";

function Profile() {
  const userName = useContext(UserContext);

  return <h2>Hello {userName}</h2>;
}

export default Profile;
```

The result is "Hello Vijesh" — notice that Profile was never given a name prop directly; the component reads the value straight from Context.

**Context API with Multiple Values**

A Provider can share more than one value at once:

```
<UserContext.Provider
  value={{
    name: "Vijesh",
    age: 16,
    course: "Computer Science"
  }}
>
```

A consuming component then reads each field from the object it receives:

```
const user = useContext(UserContext);

<p>Name: {user.name}</p>
<p>Age: {user.age}</p>
<p>Course: {user.course}</p>
```

**Combining useState() and Context API**

State management becomes more powerful once useState() and the Context API are combined — useState() stores the state, and Context shares it with multiple components:

```
useState()
    |  Stores state
    ▼
  Context
    |  Shares state
    ▼
Multiple Components
```

A Provider component that owns and shares state:

```
import { createContext, useState } from "react";

const StudentContext = createContext();

function StudentProvider({ children }) {
  const [studentName, setStudentName] = useState("Vijesh");

  return (
    <StudentContext.Provider value={{ studentName, setStudentName }}>
      {children}
    </StudentContext.Provider>
  );
}

export { StudentContext, StudentProvider };
```

A consumer that both reads and updates the shared state:

```
import { useContext } from "react";
import { StudentContext } from "./StudentContext";

function Profile() {
  const { studentName, setStudentName } = useContext(StudentContext);

  return (
    <div>
      <h2>Student Name: {studentName}</h2>
      <button onClick={() => setStudentName("Amit")}>
        Change Name
      </button>
    </div>
  );
}

export default Profile;
```

Wrapping the application in the Provider makes the shared state available anywhere inside it:

```
import { StudentProvider } from "./StudentContext";
import Profile from "./Profile";

function App() {
  return (
    <StudentProvider>
      <h1>Student Management System</h1>
      <Profile />
    </StudentProvider>
  );
}

export default App;
```

## 6. useState() vs Context API — When to Use Each

| Situation | Use |
|---|---|
| A component mainly needs to manage its own data (e.g. a counter). | useState() |
| Many components, at different levels, need the same data (e.g. logged-in user, theme). | Context API |
| Data needs to be created, changed, and shared with many components. | useState() + Context API together |

*One-line memory trick — useState() manages the data; Context API shares the data.*

## 7. Separation of Concerns (SoC) and Modular Design

In monolithic web applications, all logic (HTML, CSS, JavaScript) tends to live in a single large file. Modern component-driven design instead compartmentalises responsibilities into isolated, single-purpose components:

```
[ Global State ] ─── MemberContext.jsx (Data Hub)
        │
  ┌─────┼──────────────────┐
  ▼     ▼                  ▼
RegistrationForm      MemberCard          MemberList
(Data Entry)           (Badge View)       (Table View)
```

- Maintainability: modifying validation rules inside RegistrationForm.jsx does not affect the UI rendering of MemberCard.jsx.
- Reusability: components can be rendered independently or reused across different parts of the application tree.

- Predictable Execution Flow: the application entry (main.jsx), tree layout (App.jsx), state provider (MemberContext.jsx), and view layers each operate with clear functional boundaries.

## 8. Local State vs. Global State

| | Local State | Global State |
|---|---|---|
| Definition | State restricted to a single component that does not need to be exposed to parent or sibling components. | Data shared across multiple components located at different levels of the component hierarchy. |
| Used in this experiment for | formData and errors inside RegistrationForm.jsx, managed with useState. Sibling components do not need real-time access to every keypress before submission. | The members array and activeMember object inside MemberContext.jsx. When a registration occurs, both MemberCard and MemberList must automatically reflect the update. |

## 9. React Hooks in Detail

**useState — Immutability Principle**

```
const [state, setState] = useState(initialValue);
```

React state must never be mutated directly (for example, members.push(newMember) is invalid). State updates must supply a new object or array reference through the state updater function, e.g. setMembers(prev => [...prev, newMember]).

*Why reference immutability matters: React performs shallow reference checks to detect state changes. Direct mutation preserves the original memory reference, so React cannot tell that anything changed and will not re-render the DOM.*

**useContext — Subscription Mechanics**

```
const value = useContext(MyContext);
```

Calling useContext inside a component establishes a direct subscription to the provider. Any change to the provider's value automatically triggers a re-render of every consumer component.

## 10. Form Handling and Controlled Components

An uncontrolled component lets the DOM element keep its own internal state (accessed manually via a ref or document.getElementById()). A controlled component instead uses React state as the single source of truth for the input's value:

```
<input
  type="text"
  name="fullName"
```

```
  value={formData.fullName}   // Bound to React State
  onChange={handleChange}     // Updates React State on every keypress
/>
```

Rather than writing a separate handler for every field, JavaScript's computed property names allow one handler to update the correct field dynamically, based on the input's name attribute:

```
const handleChange = (e) => {
  const { name, value } = e.target;
  setFormData({
    ...formData,   // copies the existing fields
    [name]: value  // overwrites just the changed field
  });
};
```

### 11. Client-Side Form Validation

```
User Submits Form
        │
        ▼
e.preventDefault()   ← prevents a full page reload
        │
        ▼
     validate()
  ├── Full Name empty?  ──► append error
  ├── Roll No empty?    ──► append error
  └── Invalid Email?    ──► append error
        │
        ├── Errors exist  ──► update local 'errors' state, re-render with messages
        └── No errors     ──► call addMember(), reset form state, clear errors
```

- e.preventDefault() overrides the default HTML form behaviour, which would otherwise perform an HTTP GET/POST request and reload the page, clearing all in-memory state.
- Regex validation: a standard email-format regular expression validates the email field before it is allowed into state.
- Conditional UI rendering: a short-circuit expression displays a validation message only when an error exists, e.g. {errors.fullName && {errors.fullName}}.

## Implementation

### Directory Structure

```
aiml-club-portal/
├── index.html
└── src/
    ├── main.jsx
```

```
    ├── App.jsx
    ├── App.css
    ├── context/
    │   └── MemberContext.jsx
    └── components/
        ├── RegistrationForm.jsx
        ├── MemberCard.jsx
        └── MemberList.jsx
```

## Sequence of Execution and Data Flow

1. index.html — the entry page HTML document, defining the mount point ( ).
2. src/main.jsx — the bootstrapping module executed by the build tool (Vite). Mounts the primary App node into the DOM root.
3. src/context/MemberContext.jsx — instantiates the global context (MemberContext) and wraps descendants with MemberProvider, initializing the global state: a members array and an activeMember object.
4. src/App.jsx — the high-level container component, wrapped in MemberProvider. Houses RegistrationForm, MemberCard, and MemberList.
5. src/components/RegistrationForm.jsx — a local form component managing its own form state and validation via useState. On a valid submission, it invokes addMember() exposed by MemberContext.
6. src/components/MemberCard.jsx and src/components/MemberList.jsx — consumer components that re-render automatically via useContext whenever the context state changes.

## Code Implementation

### Step 1 — src/context/MemberContext.jsx

Handles global state management using the React Context API — this is the data hub for the whole application.

```
import { createContext, useContext, useState } from 'react';

// 1. Create Context
const MemberContext = createContext();

// 2. Provider Component
export const MemberProvider = ({ children }) => {
  const [members, setMembers] = useState([]);
  const [activeMember, setActiveMember] = useState(null);

  const addMember = (newMember) => {
    setMembers((prevMembers) => [...prevMembers, newMember]);
    setActiveMember(newMember);
  };

  return (
```

```
    <MemberContext.Provider value={{ members, activeMember, addMember }}>
      {children}
    </MemberContext.Provider>
  );
};

// 3. Custom Hook for consuming Context easily
export const useMemberContext = () => useContext(MemberContext);
```

***Explanation***

- createContext() instantiates a central context object — think of it as an empty broadcast channel that subscriber components connect to.
- MemberProvider is a wrapper component that receives its nested components as the children prop, so anything rendered inside <MemberProvider> can access the global state.
- Two pieces of state are declared: members (the full directory of registered students, starting as an empty array) and activeMember (the most recently registered student, starting as null since no one has registered yet).
- addMember() follows the immutability principle: rather than mutating the members array directly, it calls setMembers with a function that spreads the previous members and appends the new one, then calls setActiveMember(newMember) so subscriber components instantly see the newest registration.
- The Provider's value prop — { members, activeMember, addMember } — is exactly what gets broadcast to every component wrapped inside it.
- useMemberContext is a small custom hook that wraps useContext(MemberContext), so consuming components only need one import instead of two.

**Step 2 — src/components/RegistrationForm.jsx**

Handles the registration form's local state, input handling, and client-side validation.

```
import { useState } from 'react';
import { useMemberContext } from '../context/MemberContext';

export const RegistrationForm = () => {
  const { addMember } = useMemberContext();

  const [formData, setFormData] = useState({
    fullName: '',
    rollNo: '',
    email: '',
    domain: 'Computer Vision',
  });

  const [errors, setErrors] = useState({});

  const validate = () => {
    let tempErrors = {};
```

```
  if (!formData.fullName.trim()) tempErrors.fullName = 'Full Name is required.';
  if (!formData.rollNo.trim()) tempErrors.rollNo = 'Roll Number is required.';

  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!formData.email) {
    tempErrors.email = 'Email is required.';
  } else if (!emailRegex.test(formData.email)) {
    tempErrors.email = 'Invalid Email format.';
  }

  setErrors(tempErrors);
  return Object.keys(tempErrors).length === 0;
};

const handleChange = (e) => {
  const { name, value } = e.target;
  setFormData({ ...formData, [name]: value });
};

const handleSubmit = (e) => {
  e.preventDefault();
  if (validate()) {
    addMember(formData);
    setFormData({ fullName: '', rollNo: '', email: '', domain: 'Computer Vision' });
    setErrors({});
  }
};

return (
  <div className="card form-container">
    <h2>AIML Club Registration</h2>
    <form onSubmit={handleSubmit}>
      <div className="form-group">
        <label>Full Name:</label>
        <input
          type="text"
          name="fullName"
          value={formData.fullName}
          onChange={handleChange}
        />
        {errors.fullName && <span className="error">{errors.fullName}</span>}
      </div>

      <div className="form-group">
        <label>Roll Number:</label>
        <input
          type="text"
          name="rollNo"
          value={formData.rollNo}
          onChange={handleChange}
        />
        {errors.rollNo && <span className="error">{errors.rollNo}</span>}
```

```
        </div>

        <div className="form-group">
          <label>Email ID:</label>
          <input
            type="text"
            name="email"
            value={formData.email}
            onChange={handleChange}
          />
          {errors.email && <span className="error">{errors.email}</span>}
        </div>

        <div className="form-group">
          <label>Domain Interest:</label>
          <select name="domain" value={formData.domain} onChange={handleChange}>
            <option value="Computer Vision">Computer Vision</option>
            <option value="Natural Language Processing">Natural Language Processing</option>
            <option value="Generative AI">Generative AI</option>
            <option value="Robotics & RL">Robotics & RL</option>
          </select>
        </div>

        <button type="submit" className="btn">Register Member</button>
      </form>
    </div>
  );
};
```

***Explanation***

- formData and errors are local state, created with useState — they only matter to this component while the user is filling in the form.
- handleChange is a single generic handler: it reads name and value off the event target and updates just that field in formData using the spread operator, so one function handles every input.
- validate() checks that Full Name and Roll Number are non-empty, and tests the Email field against a standard email-format regular expression, collecting any problems into a tempErrors object.
- handleSubmit() calls e.preventDefault() to stop the page reloading, runs validate(), and — only if there were no errors — calls the shared addMember() from Context, then resets the local form and error state.
- Each field conditionally renders its own error message directly beneath it, using a short-circuit expression such as {errors.fullName && {errors.fullName}}.

**Step 3 — src/components/MemberCard.jsx**

Reads the shared activeMember from Context and renders it as a formatted identity card.

```
import { useMemberContext } from '../context/MemberContext';
```

```
export const MemberCard = () => {
  const { activeMember } = useMemberContext();

  return (
    <div className="card id-card-container">
      <h2>Generated Identity Card</h2>
      {activeMember ? (
        <div className="id-card">
          <div className="id-header">
            <h3>AIML STUDENT CLUB</h3>
            <p>Official Membership</p>
          </div>
          <div className="id-body">
            <p><strong>Name:</strong> {activeMember.fullName}</p>
            <p><strong>Roll No:</strong> {activeMember.rollNo}</p>
            <p><strong>Email:</strong> {activeMember.email}</p>
            <p><strong>Domain:</strong> <span
className="badge">{activeMember.domain}</span></p>
          </div>
        </div>
      ) : (
        <p className="placeholder">Fill out and submit the form to generate an ID card.</p>
      )}
    </div>
  );
};
```

***Explanation***

- useMemberContext() pulls activeMember straight out of global Context — this component never receives it as a prop.
- Conditional rendering shows a placeholder message ("Fill out and submit the form to generate an ID card.") until a member has actually been registered, and swaps to the identity-card markup once activeMember is set.
- Because this component subscribes to Context, it re-renders automatically the moment RegistrationForm calls addMember() — no manual refresh or prop passing is required.

**Step 4 — src/components/MemberList.jsx**

Reads the shared members array from Context and renders it as a directory table.

```
import { useMemberContext } from '../context/MemberContext';

export const MemberList = () => {
  const { members } = useMemberContext();

  return (
    <div className="card member-list-container">
      <h2>Registered Members ({members.length})</h2>
      {members.length === 0 ? (
        <p className="placeholder">No registered members found.</p>
```

```
      ) : (
        <table>
          <thead>
            <tr>
              <th>#</th>
              <th>Name</th>
              <th>Roll No</th>
              <th>Domain</th>
            </tr>
          </thead>
          <tbody>
            {members.map((m, index) => (
              <tr key={index}>
                <td>{index + 1}</td>
                <td>{m.fullName}</td>
                <td>{m.rollNo}</td>
                <td>{m.domain}</td>
              </tr>
            ))}
          </tbody>
        </table>
      )}
    </div>
  );
};
```

***Explanation***

- useMemberContext() supplies the full members array from global state.
- A placeholder row is shown when the array is empty; otherwise members.map() renders one table row per registered student, using the array index as the React key.
- Like MemberCard, this component re-renders automatically whenever the global members array changes.

**Step 5 — src/App.jsx**

The root component — wraps the whole tree in MemberProvider and lays out the three UI components.

```
import { MemberProvider } from './context/MemberContext';
import { RegistrationForm } from './components/RegistrationForm';
import { MemberCard } from './components/MemberCard';
import { MemberList } from './components/MemberList';
import './App.css';

function App() {
  return (
    <MemberProvider>
      <div className="app-container">
        <header className="header">
          <h1>AIML Student Club Identity Portal</h1>
        </header>
```

```
        <div className="main-content">
          <RegistrationForm />
          <MemberCard />
        </div>
        <MemberList />
      </div>
    </MemberProvider>
  );
}

export default App;
```

***Explanation***

- The entire application is wrapped in <MemberProvider>, so every component rendered inside it — RegistrationForm, MemberCard, and MemberList — can call useMemberContext() to reach the same shared state.
- App.jsx contains no business logic of its own; its only responsibility is composing the layout, in keeping with the Separation of Concerns principle.

**Step 6 — src/main.jsx**

The application's entry point — mounts the React component tree into the HTML page.

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

***Explanation***

- ReactDOM.createRoot(...).render(...) finds the #root element in index.html and tells React to take over that DOM node.
- <React.StrictMode> is a development-only helper that highlights potential problems in the application; it renders nothing visible and does not affect production behaviour.

**Step 7 — src/App.css**

Global styling for the layout, form, identity card, and member table.

```
body {
  font-family: Arial, sans-serif;
  background-color: #f4f6f9;
  margin: 0;
  padding: 20px;
}
```

```
.app-container {
  max-width: 900px;
  margin: 0 auto;
}

.header {
  text-align: center;
  margin-bottom: 20px;
  color: #1e293b;
}

.main-content {
  display: flex;
  gap: 20px;
  margin-bottom: 20px;
}

.card {
  background: white;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
  flex: 1;
}

.form-group {
  margin-bottom: 12px;
}

.form-group label {
  display: block;
  margin-bottom: 4px;
  font-weight: bold;
}

.form-group input, .form-group select {
  width: 100%;
  padding: 8px;
  box-sizing: border-box;
  border: 1px solid #ccc;
  border-radius: 4px;
}

.error {
  color: #dc2626;
  font-size: 0.85rem;
}

.btn {
  background: #2563eb;
  color: white;
```

```
  padding: 10px 15px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  width: 100%;
}

.id-card {
  border: 2px dashed #2563eb;
  border-radius: 8px;
  padding: 15px;
  background: #eff6ff;
}

.id-header {
  border-bottom: 1px solid #cbd5e1;
  text-align: center;
  margin-bottom: 10px;
}

.badge {
  background: #2563eb;
  color: white;
  padding: 2px 8px;
  border-radius: 12px;
  font-size: 0.85rem;
}

table {
  width: 100%;
  border-collapse: collapse;
}

th, td {
  border: 1px solid #e2e8f0;
  padding: 8px;
  text-align: left;
}

th {
  background-color: #f1f5f9;
}

.placeholder {
  color: #64748b;
  font-style: italic;
}
```

***Explanation***

- .main-content uses Flexbox to place the registration form and identity card side by side.
- .card gives every panel (the form, the ID card, and the member list) a consistent white

background, padding, rounded corners, and shadow.

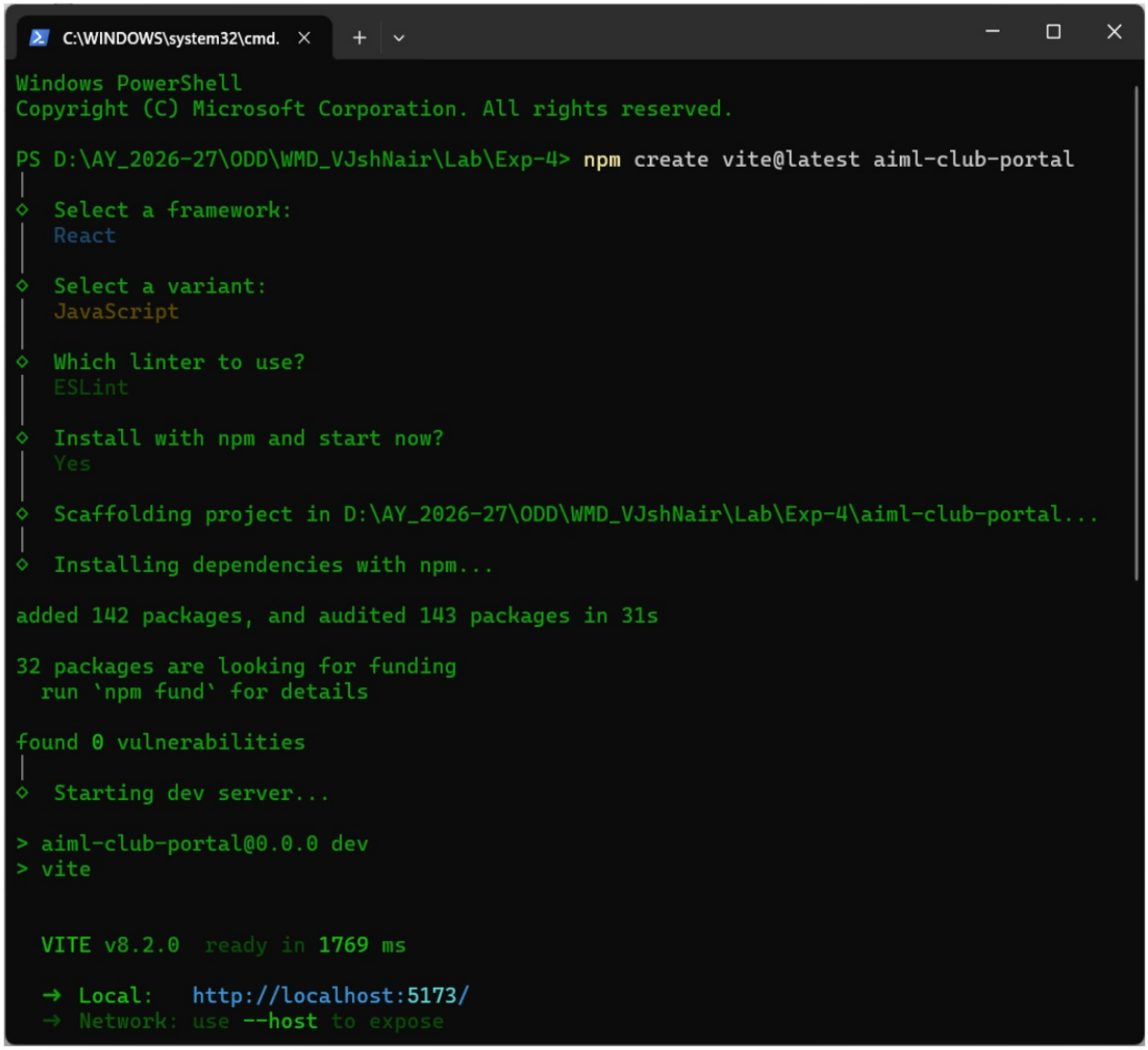
- .id-card and .badge style the generated identity card and its domain tag, while table, th, and td style the member directory.

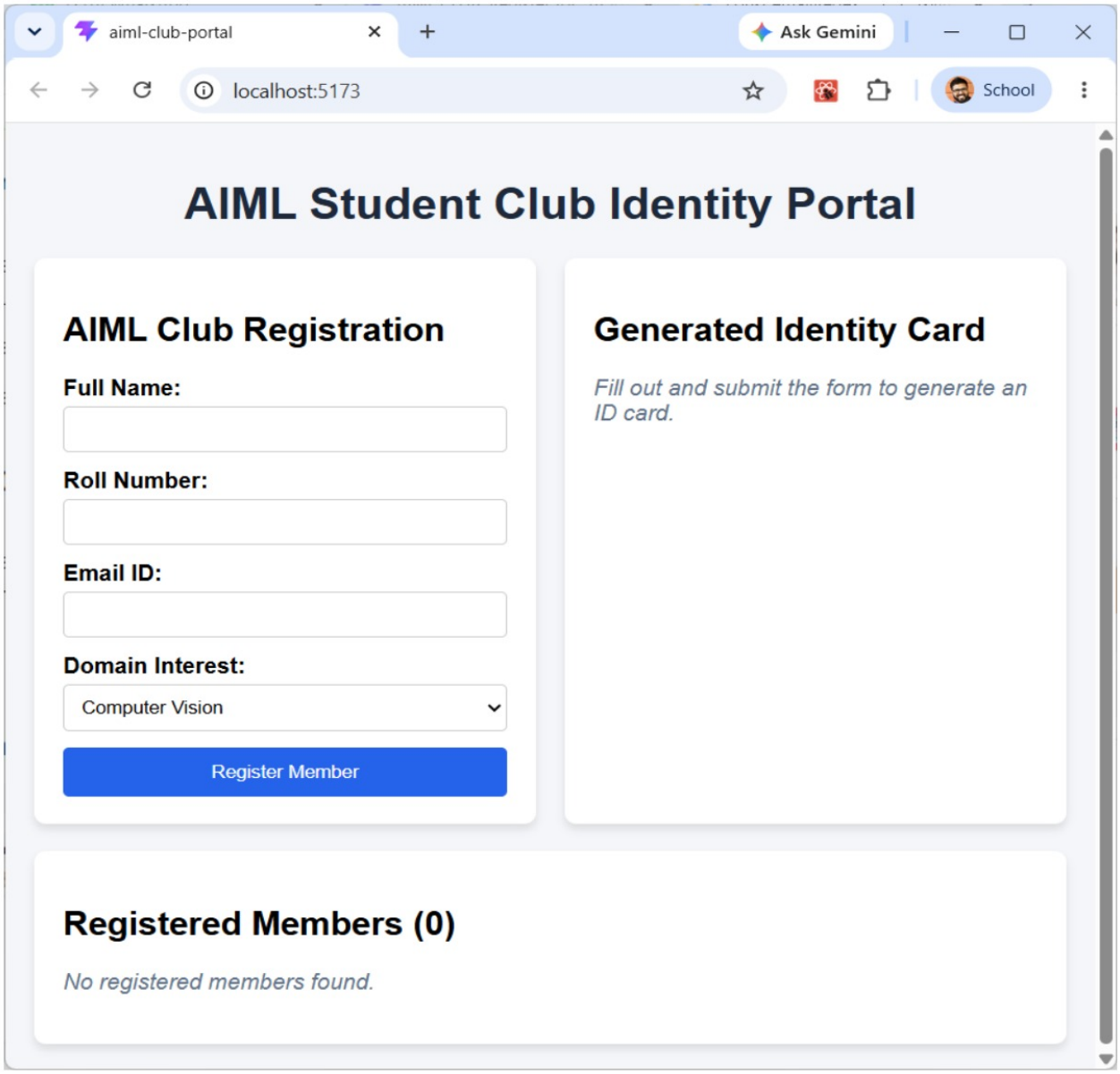
## Execution Summary and Component Lifecycle

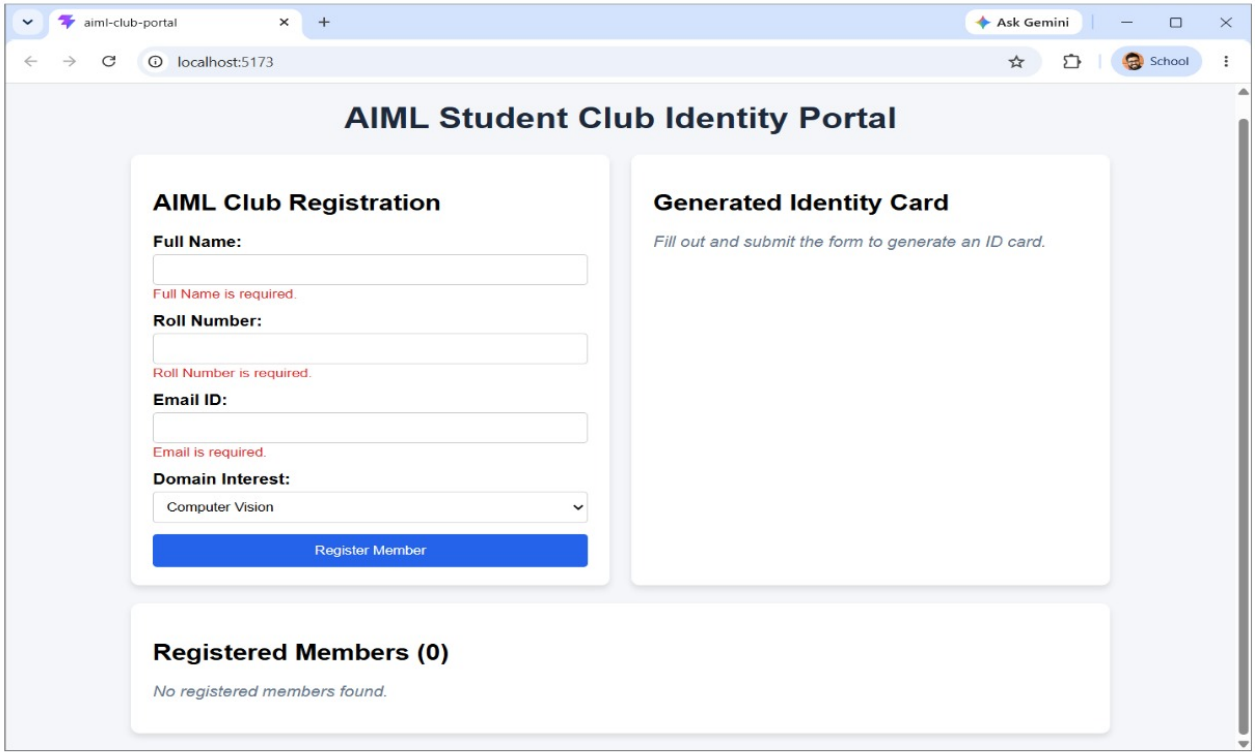
| Phase | Executed Code | Internal Mechanism |
|---|---|---|
| 1. Initialization | main.jsx | Mounts the root component into the HTML container . |
| 2. Context Setup | MemberContext.jsx | MemberProvider initializes state: members = [] and activeMember = null. |
| 3. UI Rendering | App.jsx | Renders RegistrationForm, MemberCard, and MemberList. |
| 4. User Interaction | RegistrationForm.jsx | Input changes trigger onChange events, updating local formData state via setFormData. |
| 5. Validation & Dispatch | RegistrationForm.jsx | User submits the form. validate() checks the inputs; if valid, addMember(formData) updates Context state. |
| 6. Global State Update | MemberContext.jsx | setMembers appends the new record; setActiveMember updates the target student reference. |
| 7. Subscriber Re-render | MemberCard.jsx, MemberList.jsx | Both detect the Context value change via useContext and re-render to display the updated ID Card and Member Table. |

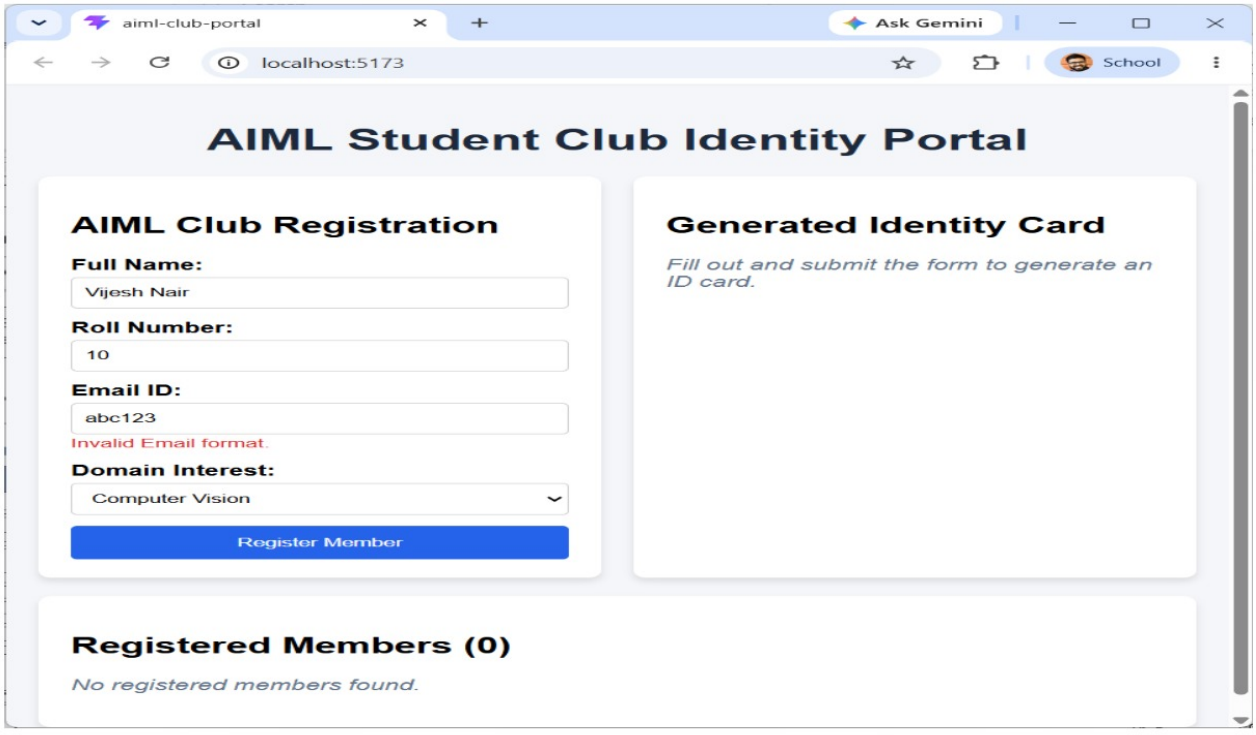
**Output:**

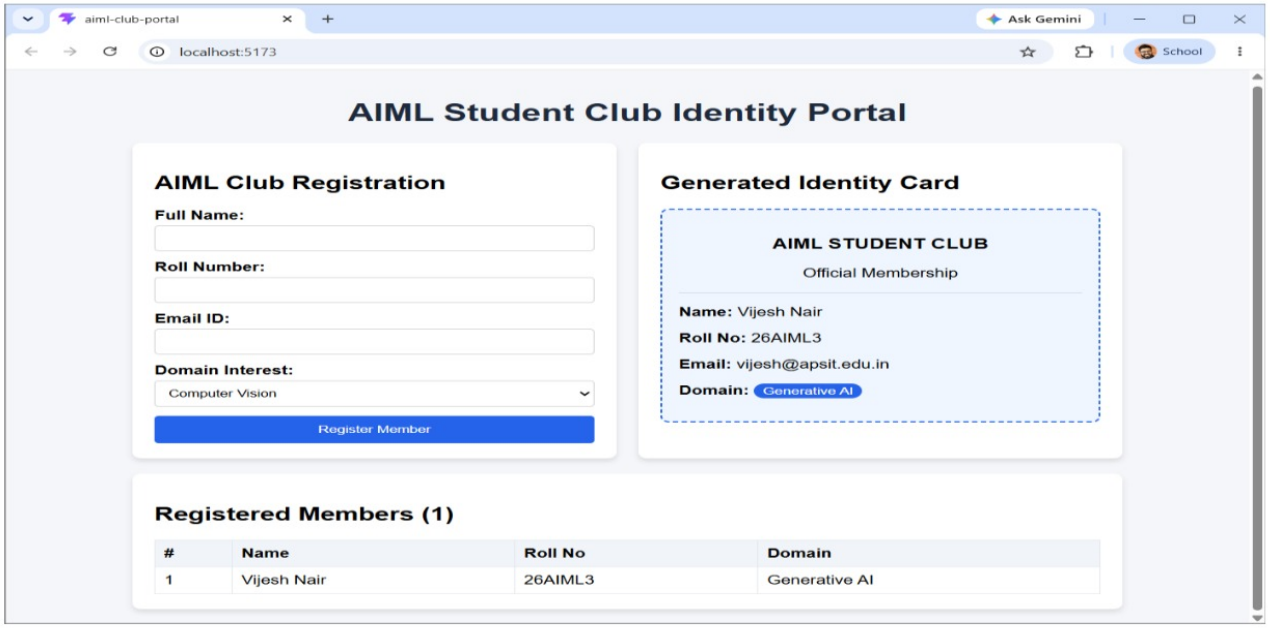
## Expected Test Cases and Application Outputs

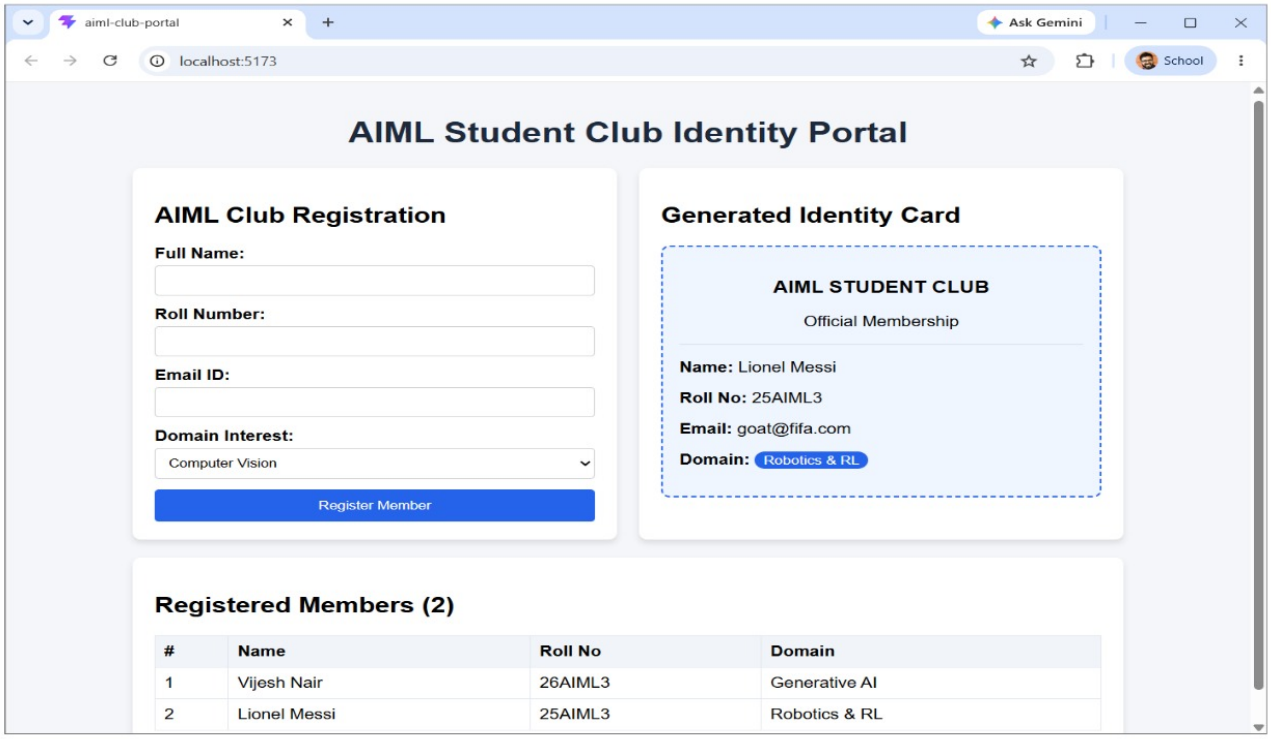

C:\WINDOWS\system32\cmd.
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
PS D:\AY_2026-27\ODD\WMD_VJshNair\Lab\Exp-4> npm create vite@latest aiml-club-portal
◇ Select a framework:
React
◇ Select a variant:
JavaScript
◇ Which linter to use?
ESLint
◇ Install with npm and start now?
Yes
◇ Scaffolding project in D:\AY_2026-27\ODD\WMD_VJshNair\Lab\Exp-4\aiml-club-portal...
◇ Installing dependencies with npm...
added 142 packages, and audited 143 packages in 31s
32 packages are looking for funding
run `npm fund` for details
found 0 vulnerabilities
◇ Starting dev server...
> aiml-club-portal@0.0.0 dev
> vite
VITE v8.2.0 ready in 1769 ms
→ Local: http://localhost:5173/
→ Network: use --host to expose


aiml-club-portal
Ask Gemini
localhost:5173
School
AIML Student Club Identity Portal
AIML Club Registration
Full Name:
Roll Number:
Email ID:
Domain Interest:
Computer Vision
Register Member
Generated Identity Card
Fill out and submit the form to generate an ID card.
Registered Members (0)
No registered members found.


aiml-club-portal
Ask Gemini
localhost:5173
School
AIML Student Club Identity Portal
AIML Club Registration
Full Name:
Full Name is required.
Roll Number:
Roll Number is required.
Email ID:
Email is required.
Domain Interest:
Computer Vision
Register Member
Generated Identity Card
Fill out and submit the form to generate an ID card.
Registered Members (0)
No registered members found.


aiml-club-portal
Ask Gemini
localhost:5173
School
AIML Student Club Identity Portal
AIML Club Registration
Full Name:
Vijesh Nair
Roll Number:
10
Email ID:
abc123
Invalid Email format.
Domain Interest:
Computer Vision
Register Member
Generated Identity Card
Fill out and submit the form to generate an ID card.
Registered Members (0)
No registered members found.




**Conclusion: -**

Thus, we have designed and implemented state management using Hooks and Context API; added form handling and validation.